

PITCH DECK

SpaceFinance

DePIN Node Financing on Creditcoin CC3

Ethereum Sepolia

Creditcoin CC3

Attestcoin USC

PROBLEM

Node operators are capital-rich, liquidity-poor

Spacecoin node operators invest real capital to run satellite ground station infrastructure.

01

Gradual revenue

Revenue arrives gradually via on-chain service fees.

02

Capital locked

Capital is locked in hardware. No liquidity.

03

Ignored by TradFi

Traditional finance ignores DePIN operators.

04

Ignored by DeFi

Existing DeFi lending ignores real-world revenue streams.

SOLUTION

Unlock liquidity without selling the asset

SpaceFinance lets node operators unlock liquidity without selling their assets.

BORROW

Lock ETH on Sepolia

→

USC proof

→

Receive mUSDF loan on CC3

REPAY

Repay from node revenue

→

USC Write-ability

→

Collateral released
automatically

No bridges. No centralized oracles. No manual admin step. Verified by Attestcoin Protocol, both directions.

Why Only Creditcoin

Remove one primitive, and the entire product falls apart:

A

Attestcoin Protocol (USC verify + Write-ability)

Trustless cross-chain proof in both directions, no bridge required.

B

Creditcoin CC3

Where Spacecoin node revenue lives (ReceiptClaimed events).

C

mUSDF

Stablecoin native to the CC3 ecosystem.

This product **cannot exist** on any other chain.

Bidirectional Two-Chain Design

ETHEREUM SEPOLIA

- CollateralVault
- NodeRegistry
- Chainlink ETH/USD
- Inbox + EOValidator

USC Proof (~8-10 min)



CREDITCOIN CC3

- SpaceFinance.execute()
- mUSDF disbursed
- Outbox

USC Write-ability message



Live contracts on both chains. Full e2e verified, both directions.

USC Integration

USC `verify()` called at 2 points:

01

Node registration proof

Sepolia NodeRegistry event

02

Collateral lock proof

Sepolia Deposited event (custom event, not WETH)

- > EvmV1Decoder deployed on-chain, linked into SpaceFinance contract.
- > Repayment confirmation runs through USC Write-ability, not USC `verify` – see next page.

Closing the Loop: CC3 → Sepolia

Most cross-chain designs in this ecosystem stop at Sepolia → CC3. SpaceFinance implements the Write-ability layer to send messages back.

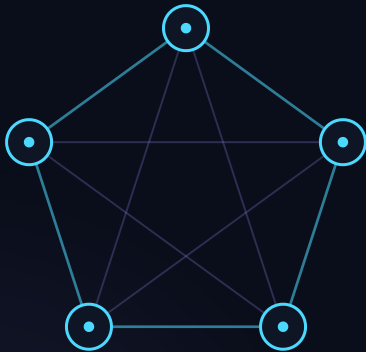


On `repay()`, SpaceFinance publishes a message through Outbox. `E0AVa`lida`t`or checks the attestor signature. Inbox delivers it to `CollateralVault`, which releases collateral automatically. No admin step.

```
> Attestor signature layer is a stub (Hardhat test wallets). Write-ability attestor infrastructure isn't yet live on public testnet – the message-passing, validation, and receiver logic are fully real, matching Creditcoin's official IMessageReceiver interface.  
> Committed 2026-08-27, ahead of the extended deadline.
```

GuildPool

Inspired by the Grameen Bank microfinance model.



5

5-node operator mutual accountability pool

Phase 1 skeleton: members are owner-vetted off-chain before approval.

→

Designed to lower collateral requirements

Once wired into SpaceFinance's LTV calculation. Not yet connected on-chain (Phase 2).

→

On-chain revenue-gated admission

Planned for Phase 2, replacing today's manual approval step.

TRACTION

Traction

- ✓ 4 core smart contracts + USC Write-ability layer (Outbox, Inbox, EOAValidator)
- ✓ Bidirectional end-to-end flow verified on-chain: Sepolia ↔ CC3
- ✓ 75 automated Hardhat tests, CI passing
- ✓ Dynamic LTV via Chainlink ETH/USD
- ✓ Full React frontend live at spacefi.vercel.app
- ✓ Demo video: youtu.be/eSDE0DheM3w

Roadmap

PHASE 1

CURRENT

Testnet

Bidirectional flow live on testnet, both directions automated.

PHASE 2

Production + GuildPool

Replace stub attester with production Write-ability set; wire GuildPool into LTV; on-chain revenue-gated admission; batch verification.

PHASE 3

Live Revenue

Live Spacecoin ReceiptClaimed revenue integration.

PHASE 4

Mainnet

Mainnet deployment on Creditcoin CC3.

TEAM

Team

OxZero (pplmaverick)

Independent on-chain builder, Taiwan

Solidity

Hardhat

React

TypeScript

Rust

GITHUB github.com/pplmaverick

TWITTER [@SmsmSmsm87](https://twitter.com/SmsmSmsm87)

DEMO spacefi.vercel.app